PHP

Session variables

MCAST

INTRODUCTION

- A session variable in PHP is a way to store user-specific information that persists across multiple pages during a user's visit to a website.

- Session variables are stored on the server-side and are accessible throughout the user's session.

SESSION INITIALIZATION

- A session needs to be started. When a session starts, PHP generates a unique session ID, typically a 32-character string of hexadecimal numbers.

- A session is started using the **session_start()** function at the beginning of your script to initialize or resume a session.

- This session ID is usually stored in a cookie called PHPSESSID on the user's browser.

DATA STORAGE

- Session data is stored in a temporary file on the server.
- The location of this file on the server is determined by the "session.save_path" setting in the php.ini configuration file.
- In PHP, Session variables are stored in the **$_SESSION** superglobal array.
- Each session variable is a key-value pair in this array.

DATA STORAGE

- Session data is stored in a temporary file on the server.
- The location of this file on the server is determined by the "session.save_path" setting in the php.ini configuration file.
- In PHP, Session variables are stored in the **$_SESSION** superglobal array.
- Each session variable is a key-value pair in this array.

ACCESSING SESSION DATA

- Data is set using the $_SESSION super global array.
- Data is retrieved using the $_SESSION super global array.

```php
<?php
$_SESSION["username"] = "JohnDoe";
$_SESSION["user_id"] = 12345;
?>
```

```php
<?php
echo $_SESSION["username"]; // Outputs: JohnDoe
?>
```

UNSETTING SESSION DATA

- To remove specific session variables use the **unset()** function.

```
<?php
unset($_SESSION["username"]);
?>
```

DESTROYING A SESSION

- To end a php session use **session_destroy()** function. It can be also used in conjunction with **session_unset()**.

```
<?php
session_unset();
session_destroy();
?>
```

KEY CONSIDERATIONS

- Each file needs to start with the **session_start()** function for any session operation using the **$_SESSION** to happen.

- It is important to destroy a session when done with php sessions using **session_destroy()**.

KEY CONSIDERATIONS

- Security: Session data is stored on the server, making it more secure than client-side storage methods like cookies.
- Concurrency: File-based sessions (PHP's default) lock the session file, which can cause issues with concurrent requests.
- Session ID: PHP generates a unique session ID for each session, typically stored in a cookie called **PHPSESSID**.
- Configuration: Session behavior can be configured in php.ini, including settings like **session lifetime** and **storage location**.

MCAST

Any Questions ?